

Homework set 4

Please submit this Jupyter notebook through Canvas no later than Monday December 9, 9:00. Submit the notebook file with your answers (as .ipynb file) and a pdf printout. The pdf version can be used by the teachers to provide feedback. On canvas there are hints about creating a nice pdf version.

Before you hand in, please make sure the notebook runs, by running "Restart kernel and run all cells..." from the Kernel menu.

Homework is in groups of two, and you are expected to hand in original work. Work that is copied from another group will not be accepted.

Exercise 0

Write down the names + student ID of the people in your group.

Format: Name [ID]

1. Kushnava Singha [15663388]
2. Shania Sinha [14379031]

Run the following cell to import NumPy, Matplotlib. If anything else is needed you can import this yourself.

```
In [46]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib
from sympy import *
from tqdm import tqdm
```

Exercise 1

N.B.1. you are to implement the methods yourself.

N.B.2. Tentative distribution of points is 2+1+2+2+2 points (plus 1 point makes 10).

Given a function f , let $T(f, a, b, m)$ denote the composite trapezoid rule with m subintervals over the interval $[a, b]$.

(a)

Approximate the integral of x^{-3} over $[a, b] = [\frac{1}{10}, 100]$ by the composite trapezoid rule $T(f, a, b, m)$ for $m = 2^k$. Plot the absolute approximation error for different values of k . Find the smallest k such that the exact error is less than $\epsilon = 10^{-3}$. Explain the slow convergence.

```
In [29]: def composite_trapezoid(f, a, b, m):
    h = (b - a) / m # Length of subintervals
    x = np.linspace(a, b, m + 1) # Subinterval end points
    y = f(x) # Function values at end points

    return (h / 2) * (y[0] + 2 * np.sum(y[1:-1]) + y[-1])

# Parameters
a = 0.1
b = 100
k = np.linspace(0, 28, 1000)
m = 2**k
tol = 1e-3 # Epsilon
```

```
In [69]: x = symbols('x')
f = Lambda(x: x**-3, x)
f_double_prime = diff(f_double_prime, x, 2) # First differential
f_double_prime = diff(f_double_prime, x, 2) # Second differential

# Get absolute approximation errors for each estimate
def error(f_double_prime, a, b, x, m):
    h = (b - a) / m
    return h**2 * integrate(f_double_prime, (x, a, b)) / 12
```

```
In [76]: # Get composite_trapezoid estimated area
estimated_area = [composite_trapezoid(f, a, b, int(value)) for value in tqdm(m)]

100%|██████████| 1000/1000 [07:15<00:00, 2.29it/s]
```

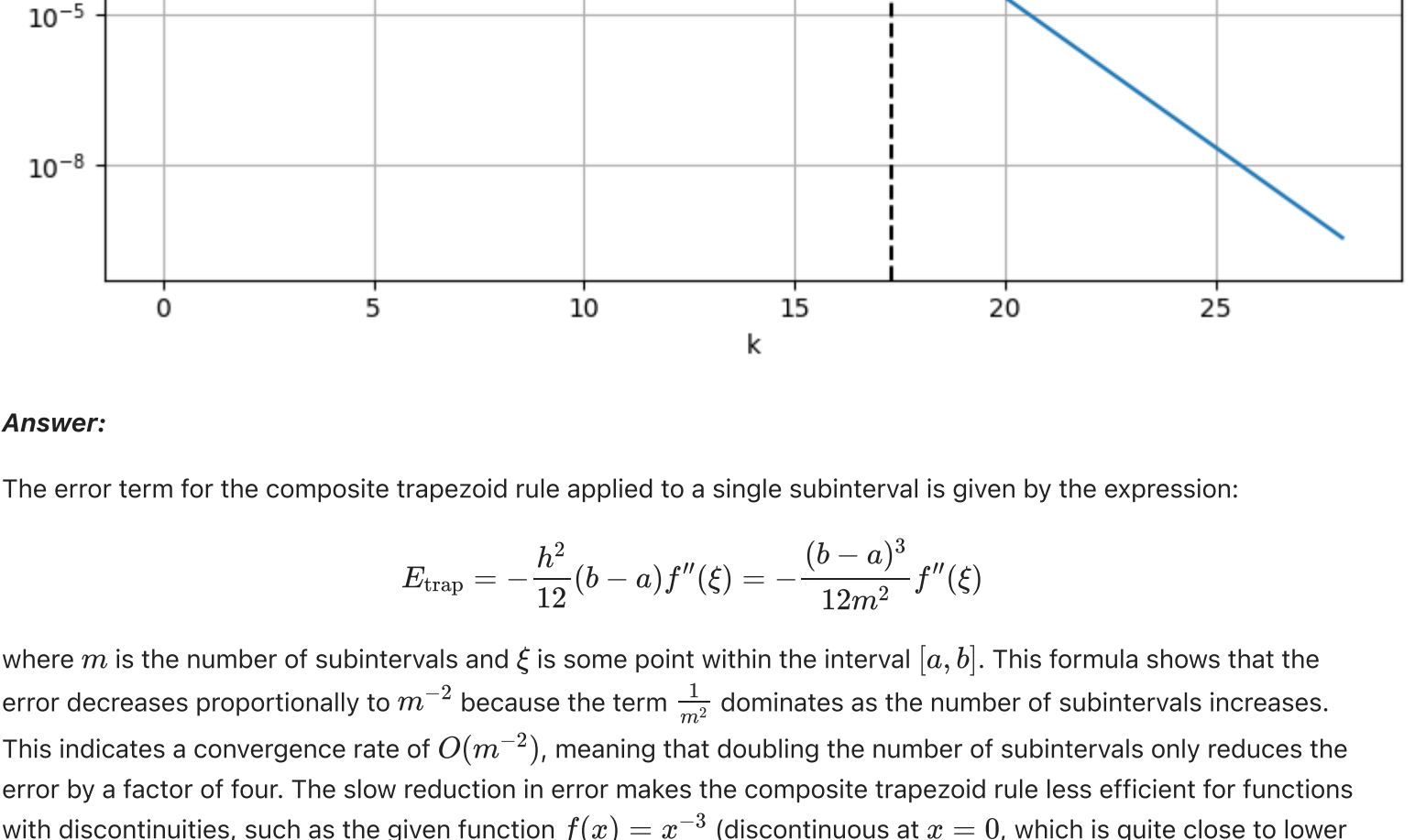
```
In [77]: # Get absolute approximation errors for each estimate
absolute_approximation_errors = [error(f_double_prime, a, b, x, value) for value in tqdm(m)]

100%|██████████| 1000/1000 [00:13<00:00, 72.94it/s]
```

```
In [78]: # The smallest value of k such that the exact error is less than tol
exact_value = integrate(x**-3, (x, a, b))
exact_error = [np.abs(area - exact_value) for area in estimated_area]
min_good_k = k[np.where(np.array(exact_error) < tol)][0]
print(f"The smallest value of k such that the exact error is less than tol is: {min_good_k:.3f}")
```

The smallest value of k such that the exact error is less than tol is: 17.293

```
In [80]: # Plot absolute error vs k
plt.figure(figsize=(8, 6))
plt.plot(k, absolute_approximation_errors, label='Absolute Approximation Error')
plt.axvline(min_good_k, color='black', linestyle='--', label='Smallest k such that exact error < \epsilon')
plt.axhline(tol, color='red', linestyle='--', label='Tolerance')
plt.yscale('log')
plt.xlabel('k')
plt.ylabel('log(Error)')
plt.title('Absolute Approximation Error vs k', fontweight='bold')
plt.legend(loc='upper right')
plt.grid()
plt.tight_layout()
plt.show()
```



Answer:

The error term for the composite trapezoid rule applied to a single subinterval is given by the expression:

$$E_{\text{trap}} = -\frac{h^2}{12}(b-a)f''(\xi) = -\frac{(b-a)^3}{12m^2}f''(\xi)$$

where m is the number of subintervals and ξ is some point within the interval $[a, b]$. This formula shows that the error decreases proportionally to m^{-2} because the term $\frac{1}{m^2}$ dominates as the number of subintervals increases. This indicates a convergence rate of $O(m^{-2})$, meaning that doubling the number of subintervals only reduces the error by a factor of four. The slow reduction in error makes the composite trapezoid rule less efficient for functions with discontinuities, such as the given function $f(x) = x^{-3}$ (discontinuous at $x = 0$, which is quite close to lower bound of the given range $a = 1/10$).

(b)

To improve the convergence rate of the above problem, we may use an adaptive strategy, as discussed in the book and the lecture. Consider the following formulas for approximate integration

$$I_1(f, a, b) = T(f, a, b, 1)$$

$$I_2(f, a, b) = T(f, a, b, 2).$$

Show, based on the precise error estimates for the trapezoid rule from the book/lecture that the error in I_2 can be estimated by a formula of the form

$$E_2 = C(I_1 - I_2)$$

and determine the constant C (if you can't find C , you may take $C = 0.5$).

Answer:

Error terms:

The error for the composite trapezoid approximation with one subinterval $[a, b]$ is given by:

$$E_1 = -\frac{h^2}{12}f''(\xi) = -\frac{(b-a)^3}{12}f''(\xi) \quad \text{for some } \xi \in [a, b]$$

Hence, the error in composite trapezoid approximation with two subintervals is:

$$E_2 = \left(\frac{(b-a)}{2}\right)^2 \frac{f''(\xi)}{12} = -\frac{(b-a)^3}{48}f''(\xi)$$

Difference in quadrature approximations:

$$I_1 - I_2 = T(f, a, b, 1) - T(f, a, b, 2)$$

Substituting the known values of the errors and simplifying, we get:

$$I_1 - I_2 = (f(a) + f(b))\frac{b-a}{2} - \frac{b-a}{4}\left(f(a) + 2f\left(\frac{a+b}{2}\right) + f(b)\right)$$

$$I_1 - I_2 = \frac{b-a}{4}\left(f(a) + f(b) - 2f\left(\frac{a+b}{2}\right)\right)$$

To obtain the difference between the approximations of the two quadratures in a form similar to that of the errors, we use Taylor expansion of the functions $f(a)$ and $f(b)$ with their midpoint as the reference. The midpoint is at $(a+b)/2$.

$$f(a) = f\left(\frac{a+b}{2}\right) - \frac{b-a}{2}f'\left(\frac{a+b}{2}\right) + \frac{(b-a)^2}{8}f''(\xi_a)$$

$$f(b) = f\left(\frac{a+b}{2}\right) + \frac{b-a}{2}f'\left(\frac{a+b}{2}\right) + \frac{(b-a)^2}{8}f''(\xi_b)$$

Substitute these expansions back into the expression for $I_1 - I_2$:

$$I_1 - I_2 = \frac{b-a}{4}\left(f(a) + f(b) - 2f\left(\frac{a+b}{2}\right)\right)$$

$$= \frac{b-a}{4}\left(\left[f\left(\frac{a+b}{2}\right) - \frac{b-a}{2}f'\left(\frac{a+b}{2}\right) + \frac{(b-a)^2}{8}f''(\xi_a)\right] + \left[f\left(\frac{a+b}{2}\right) + \frac{b-a}{2}f'\left(\frac{a+b}{2}\right) + \frac{(b-a)^2}{8}f''(\xi_b)\right] - 2f\left(\frac{a+b}{2}\right)\right)$$

$$I_1 - I_2 = \frac{b-a}{4}\left(\frac{(b-a)^2}{8}f''(\xi_a) + \frac{(b-a)^2}{8}f''(\xi_b)\right)$$

$$I_1 - I_2 = \frac{(b-a)^3}{32}(f''(\xi_a) + f''(\xi_b))$$

Finding the value of C

We assume:

$$E_2 = C \cdot (I_1 - I_2)$$

Substitute the expressions:

$$-\frac{(b-a)^3}{48}f''(\xi) = C \cdot \frac{(b-a)^3}{32}(f''(\xi_a) + f''(\xi_b))$$

Assuming $f''(\xi) \approx \frac{f''(\xi_a) + f''(\xi_b)}{2}$:

$$-\frac{1}{48}f''(\xi) \approx C \cdot \frac{1}{32} \cdot 2f''(\xi)$$

Solving for the constant C , we get:

$$-\frac{1}{48} = C \cdot \frac{1}{16}$$

$$C = -\frac{1}{48} \cdot 16 = \frac{16}{48} = -\frac{1}{3}$$

Hence, the value of C is $-\frac{1}{3}$.

(c)

An adaptive strategy for computing the integral on an interval $[a, b]$ now is: Compute I_2 and E_2 , and accept I_2 as an approximation when the estimated error E_2 is less or equal than a desired tolerance ϵ . Otherwise, apply the procedure to $\int_a^{\frac{a+b}{2}} f(x) dx$ and $\int_{\frac{a+b}{2}}^b f(x) dx$ with tolerances $\frac{\epsilon}{2}$.

Write a recursive python routine that implements the adaptive strategy.

Then apply this routine to the function x^{-3} with a, b, ϵ as before. What is the exact error in the obtained approximation?

```
In [103]: # Constant C (decimal version of the constant, rounded to 2 decimal places)
C = -0.33
```

```
In [104]: def adaptquad(f, a, b, C, tol):
    I_1 = composite_trapezoid(f, a, b, 1) # I_1: Quadrature 1
    I_2 = composite_trapezoid(f, a, b, 2) # I_2: Quadrature 2

    E_2 = C * (I_1 - I_2) # Error 2 estimate

    mid = a + (b - a) / 2 # Midpoint

    if mid <= a or mid >= b:
        warnings.warn("Error: m is not within the interval")
        return I_2

    if abs(E_2) <= tol:
        return I_2
    else:
        new_tol = tol / 2
        return adaptquad(f, a, mid, C, new_tol) + adaptquad(f, mid, b, C, new_tol)
```

```
In [105]: # Adaptive approximation
adaptive_approximation = adaptquad(f, a, b, C, tol)
print(f'Adaptive Approximation: {adaptive_approximation}')
```

```
# Exact value
exact_value = integrate(x**-3, (x, a, b))
print(f'Exact Value: {exact_value}')
```

```
# Error
exact_error = abs(adaptive_approximation - exact_value)
print(f'Exact Error: {exact_error}')
```

Adaptive Approximation: 50.00014852119522
Exact Value: 49.99995000000000
Exact Error: 0.000198521195230228

(d)

Modify the code of (c) so that the number of function evaluations is counted. Optimize your implementation such that no unnecessary function evaluations are performed.

Compare the number of function evaluations used in the adaptive strategy of (c) with the result of (a). (Hint: To count the number of function evaluations, you may use a global variable.)

```
In [97]: # Initialize a global counter for function evaluations
function_evaluations = 0
```

```
In [86]: # Updated composite trapezoid rule
def composite_trapezoid_count(f, a, b, m):
    global function_evaluations

    h = (b - a) / m # Length of subintervals
    x = np.linspace(a, b, m + 1) # Subinterval end points
    y = f(x) # Function values at end points

    function_evaluations += len(x) # Update function evaluations

    return (h / 2) * (y[0] + 2 * np.sum(y[1:-1]) + y[-1])
```

```
In [ ]: # Updated adaptive integration with evaluation counting
def adaptquad_count(f, a, b, C, tol):
    global function_evaluations

    I_1 = composite_trapezoid_count(f, a, b, 1) # I_1: Quadrature 1
    I_2 = composite_trapezoid_count(f, a, b, 2) # I_2: Quadrature 2

    E_2 = C * (I_1 - I_2) # Error 2 estimate

    mid = a + (b - a) / 2 # Midpoint

    if mid <= a or mid >= b:
        warnings.warn("Error: m is not within the interval")
        return I_2

    if abs(E_2) <= tol:
        return I_2
    else:
        new_tol = tol / 2
        return adaptquad_count(f, a, mid, C, new_tol) + adaptquad_count(f, mid, b, C, new_tol)
```

```
In [ ]: function_evaluations = 0 # Reset function evaluations

# Composite Trapezoid approximation at last value of m
trapezoid_approximation = composite_trapezoid_count(f, a, b, int(m[-1]))

# Trapezoid approximation
print(f'Composite Trapezoid Approximation: {trapezoid_approximation} with m = {m[-1]}, k = {k[-1]}')
```

```
# Function evaluations
print(f'Function Evaluations for Composite Trapezoid: {function_evaluations}')
```

Composite Trapezoid Approximation: 49.99995000034584 with m = 268435456.0, k = 28.0
Function Evaluations for Composite Trapezoid: 268435457

```
In [ ]: m_min_good_k = 2**min_good_k # m value for the smallest k such that the exact error is less than
function_evaluations = 0 # Reset function evaluations
```

```
trapezoid_approximation_min_good_k = composite_trapezoid_count(f, a, b, int(m_min_good_k))
print(f'Composite Trapezoid Approximation: {trapezoid_approximation_min_good_k} with m = {m_min_good_k}')
```

```
# Function Evaluations
print(f'Function Evaluations for Composite Trapezoid: {function_evaluations} with m = {m_min_good_k}')
```

Composite Trapezoid Approximation: 50.00014852119522 with m = 160620.142
Function Evaluations for Composite Trapezoid: 160621 with m = 160620.142

```
In [ ]: function_evaluations = 0 # Reset function evaluations

# Compute the adaptive approximation with evaluation counting
adaptive_approximation = adaptquad_count(f, a, b, C, tol)
```

```
# Adaptive approximation
print(f'Adaptive Approximation: {adaptive_approximation}')
```

```
# Function evaluations
print(f'Function Evaluations for Adaptive Approximation: {function_evaluations}')
```

Adaptive Approximation: 50.00014852119522
Function Evaluations for Adaptive Approximation: 48095

Answer:

All functions are as optimized as possible. The check condition in `adaptquad()` and `adaptquad_count()` ensures that once the midpoint is at a value outside the given range $[a, b]$, the recursion stops and the function returns the best estimate (I_2) and also displays a warning message. This ensures the recursion always returns something and does not run infinitely. We also check if the calculated error is less than the given tolerance ($\epsilon = 10^{-3}$), which is not too tight so as it keep additional function evaluations that contribute little to the accuracy of the estimate to a minimum. Having these two conditions in place removes the need of checking if the length of the interval $[a, b]$ has reduced to a value below the machine epsilon.

The composite trapezoid quadrature evaluates the value of a function 268,435,457 times when $k = 28$. The number of function evaluations is 160,621 when $k = 17.23$, the minimum value of k at which the exact error is less than the tolerance ($\epsilon = 10^{-3}$).

The adaptive quadrature computes a lot less function evaluations than the regular composite trapezoid rule, as is expected. It makes 48,095 evaluations.

(e)

In the course of executing the recursive procedure, some subintervals are refined (split into two subintervals) while others aren't as a result of the choices made by the algorithm. It turns out that the choices made by this algorithm are not always optimal. Other algorithms, that decide in a different way which subinterval needs to be refined, may be more efficient in the sense that they require less function evaluations (while using the same formulas for the approximate integral and the approximate error associated with a subinterval).

Can you explain why this is the case? Devise an alternative, non-recursive algorithm that addresses this issue and should lead to a more efficient integral computation. Describe your approach and algorithm in about 5 to 10 sentences (bullet points).

Answer:

The recursive adaptive quadrature algorithm can be inefficient due to how it makes local decisions. Each subinterval is treated independently, and the algorithm does not consider the global errors of the complete integration domain. This can lead to unnecessary refinement of less important intervals while neglecting the more critical areas.

Another limitation comes from fixed tolerance splitting. When the algorithm recurses, it halves the tolerance for each subinterval. This approach can result in excessive calculations, especially in regions where the function is already well-approximated. This issue can be mitigated by implementing a procedure such that once the algorithm has sufficiently refined a subinterval, it is never revisited. A smarter recursion routine will reduce the computational cost.

Alternative algorithm:

We can use an iterative "refinement loop" instead of recursion in the adaptive quadrature method. This method avoids deep recursion and allows for better memory management.

- Start with the entire integration interval [a, b].
- Initialize a list `intervals` containing the starting interval (a, b).
- Initialize the variable `total_integral = 0.0`.
- While the list `intervals` is not empty and the iteration count is below a maximum threshold:
 - Extract the first interval (left, right) from the list and compute its midpoint `mid = (left + right) / 2`.
 - Apply the composite trapezoid rule with one and two subintervals to the extracted interval.
 - Compute the error estimate: $E_2 = C(I_1 - I_2)$, where I_1 and I_2 are computed using one and two subintervals respectively.
 - If the absolute error $|E_2|$ is less than or equal to the tolerance, add I_2 to `total_integral`.
 - If the error is larger than the tolerance, subdivide the interval and add (left, mid) and (mid, right) to the list `intervals`.
- Return `total_integral`.